

Assignment 3:

Q1.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct Node {  
    struct Node *lchild;  
    int data;  
    struct Node *rchild;  
} Node;
```

```
typedef struct Queue {  
    int size;  
    int front;  
    int rear;  
    Node **Q;  
} Queue;
```

```
void createQueue(Queue *q, int size) {  
    q->size = size;  
    q->front = q->rear = 0;  
    q->Q = (Node **)malloc(q->size * sizeof(Node *));  
}
```

```
void enqueue(Queue *q, Node *x) {  
    if ((q->rear + 1) % q->size == q->front)  
        printf("Queue is Full\n");  
    else {  
        q->rear = (q->rear + 1) % q->size;  
        q->Q[q->rear] = x;  
    }  
}
```

```
Node *dequeue(Queue *q) {
    Node *x = NULL;

    if (q->front == q->rear)
        printf("Queue is Empty\n");
    else {
        q->front = (q->front + 1) % q->size;
        x = q->Q[q->front];
    }
    return x;
}
```

```
int isEmpty(Queue q) {
    return q.front == q.rear;
}
```

```
Node *root = NULL;
```

```
void createTree() {
    Node *p, *t;
    int x;
    Queue q;
    createQueue(&q, 100);

    printf("Enter root value: ");
    scanf("%d", &x);
    root = (Node *)malloc(sizeof(Node));
    root->data = x;
    root->lchild = root->rchild = NULL;
    enqueue(&q, root);
}
```

```

while (!isEmpty(q)) {
    p = dequeue(&q);
    printf("Enter left child of %d (-1 for no child): ", p->data);
    scanf("%d", &x);
    if (x != -1) {
        t = (Node *)malloc(sizeof(Node));
        t->data = x;
        t->lchild = t->rchild = NULL;
        p->lchild = t;
        enqueue(&q, t);
    }

    printf("Enter right child of %d (-1 for no child): ", p->data);
    scanf("%d", &x);
    if (x != -1) {
        t = (Node *)malloc(sizeof(Node));
        t->data = x;
        t->lchild = t->rchild = NULL;
        p->rchild = t;
        enqueue(&q, t);
    }
}

void preorder(Node *p) {
    if (p) {
        printf("%d ", p->data);
        preorder(p->lchild);
        preorder(p->rchild);
    }
}

```

```
void inorder(Node *p) {  
    if (p) {  
        inorder(p->lchild);  
        printf("%d ", p->data);  
        inorder(p->rchild);  
    }  
}
```

```
void postorder(Node *p) {  
    if (p) {  
        postorder(p->lchild);  
        postorder(p->rchild);  
        printf("%d ", p->data);  
    }  
}
```

```
int main() {  
    int choice;  
  
    while (1) {  
        printf("\nMenu:\n");  
        printf("1. Create Binary Tree\n");  
        printf("2. In-order Traversal\n");  
        printf("3. Pre-order Traversal\n");  
        printf("4. Post-order Traversal\n");  
        printf("5. Exit\n");  
        printf("Enter your choice: ");  
        scanf("%d", &choice);  
  
        switch (choice) {  
            case 1:  
                createTree();
```

```

        break;
    case 2:
        printf("In-order Traversal: ");
        inorder(root);
        printf("\n");
        break;
    case 3:
        printf("Pre-order Traversal: ");
        preorder(root);
        printf("\n");
        break;
    case 4:
        printf("Post-order Traversal: ");
        postorder(root);
        printf("\n");
        break;
    case 5:
        exit(0);
    default:
        printf("Invalid choice! Please try again.\n");
    }
}

return 0;
}

```

q2.

```

#include <stdio.h>
#include <stdlib.h>

```

```

typedef struct Node {
    struct Node *lchild;

```

```

    int data;

    struct Node *rchild;
} Node;

typedef struct Queue {
    int size;

    int front;

    int rear;

    Node **Q;
} Queue;

void createQueue(Queue *q, int size) {
    q->size = size;

    q->front = q->rear = 0;

    q->Q = (Node **)malloc(q->size * sizeof(Node *));
}

void enqueue(Queue *q, Node *x) {
    if ((q->rear + 1) % q->size == q->front)
        printf("Queue is Full\n");
    else {
        q->rear = (q->rear + 1) % q->size;

        q->Q[q->rear] = x;
    }
}

Node *dequeue(Queue *q) {
    Node *x = NULL;

    if (q->front == q->rear)
        printf("Queue is Empty\n");
    else {

```

```

        q->front = (q->front + 1) % q->size;
        x = q->Q[q->front];
    }
    return x;
}

```

```

int isEmpty(Queue q) {
    return q.front == q.rear;
}

```

```

Node *root = NULL;

```

```

void createTree() {
    Node *p, *t;
    int x;
    Queue q;
    createQueue(&q, 100);

    printf("Enter root value: ");
    scanf("%d", &x);
    root = (Node *)malloc(sizeof(Node));
    root->data = x;
    root->lchild = root->rchild = NULL;
    enqueue(&q, root);

```

```

while (!isEmpty(q)) {
    p = dequeue(&q);
    printf("Enter left child of %d (-1 for no child): ", p->data);
    scanf("%d", &x);
    if (x != -1) {
        t = (Node *)malloc(sizeof(Node));
        t->data = x;

```

```

        t->lchild = t->rchild = NULL;

        p->lchild = t;
        enqueue(&q, t);
    }

    printf("Enter right child of %d (-1 for no child): ", p->data);
    scanf("%d", &x);
    if (x != -1) {
        t = (Node *)malloc(sizeof(Node));
        t->data = x;
        t->lchild = t->rchild = NULL;
        p->rchild = t;
        enqueue(&q, t);
    }
}

int searchInorder(int inArray[], int inStart, int inEnd, int data) {
    for (int i = inStart; i <= inEnd; i++) {
        if (inArray[i] == data) {
            return i;
        }
    }
    return -1;
}

Node *generateFromTraversal(int *inorder, int *preorder, int inStart, int inEnd) {
    static int preIndex = 0;

    if (inStart > inEnd) {
        return NULL;
    }

```

```

Node *node = (Node *)malloc(sizeof(Node));
node->data = preorder[preIndex++];
node->lchild = node->rchild = NULL;

if (inStart == inEnd) {
    return node;
}

int splitIndex = searchInorder(inorder, inStart, inEnd, node->data);
node->lchild = generateFromTraversal(inorder, preorder, inStart, splitIndex - 1);
node->rchild = generateFromTraversal(inorder, preorder, splitIndex + 1, inEnd);

return node;
}

void preorder(Node *p) {
    if (p) {
        printf("%d ", p->data);
        preorder(p->lchild);
        preorder(p->rchild);
    }
}

void inorder(Node *p) {
    if (p) {
        inorder(p->lchild);
        printf("%d ", p->data);
        inorder(p->rchild);
    }
}

```

```
void postorder(Node *p) {  
    if (p) {  
        postorder(p->lchild);  
        postorder(p->rchild);  
        printf("%d ", p->data);  
    }  
}
```

```
int main() {  
    int choice;  
    int n, *inorderArr, *preorderArr;  
  
    while (1) {  
        printf("\nMenu:\n");  
        printf("1. Create Binary Tree\n");  
        printf("2. Create Tree from Inorder and Preorder Traversals\n");  
        printf("3. Preorder Traversal\n");  
        printf("4. Inorder Traversal\n");  
        printf("5. Postorder Traversal\n");  
        printf("6. Exit\n");  
        printf("Enter your choice: ");  
        scanf("%d", &choice);  
  
        switch (choice) {  
            case 1:  
                createTree();  
                break;  
            case 2:  
                printf("Enter the number of nodes: ");  
                scanf("%d", &n);  
  
                inorderArr = (int *)malloc(n * sizeof(int));
```

```
preorderArr = (int *)malloc(n * sizeof(int));
```

```
printf("Enter inorder traversal: ");
```

```
for (int i = 0; i < n; i++) {  
    scanf("%d", &inorderArr[i]);  
}
```

```
printf("Enter preorder traversal: ");
```

```
for (int i = 0; i < n; i++) {  
    scanf("%d", &preorderArr[i]);  
}
```

```
root = generateFromTraversal(inorderArr, preorderArr, 0, n - 1);
```

```
free(inorderArr);
```

```
free(preorderArr);
```

```
break;
```

```
case 3:
```

```
printf("\nPreorder Traversal: ");
```

```
preorder(root);
```

```
printf("\n");
```

```
break;
```

```
case 4:
```

```
printf("\nInorder Traversal: ");
```

```
inorder(root);
```

```
printf("\n");
```

```
break;
```

```
case 5:
```

```
printf("\nPostorder Traversal: ");
```

```
postorder(root);
```

```
printf("\n");
```

```
break;
```

```
        case 6:
            exit(0);
        default:
            printf("Invalid choice! Please try again.\n");
    }
}

return 0;
}
```

Q3.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct Node {
    int data;
    struct Node* lchild;
    struct Node* rchild;
} Node;
```

```
Node* root = NULL;
```

```
Node* createNode(int key) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = key;
    newNode->lchild = NULL;
    newNode->rchild = NULL;
    return newNode;
}
```

```
Node* insert(Node* p, int key) {
    if (p == NULL) {
```

```

        return createNode(key);
    }

    if (key < p->data) {
        p->lchild = insert(p->lchild, key);
    } else if (key > p->data) {
        p->rchild = insert(p->rchild, key);
    }
    return p;
}

```

```

Node* search(Node* p, int key) {
    if (p == NULL || p->data == key) {
        return p;
    }
}

```

```

    if (key < p->data) {
        return search(p->lchild, key);
    } else {
        return search(p->rchild, key);
    }
}
}

```

```

Node* inOrderSuccessor(Node* root, Node* x) {
    Node* ans = NULL;
    while (root) {
        if (root->data <= x->data) {
            root = root->rchild;
        } else {
            ans = root;
            root = root->lchild;
        }
    }
}

```

```

    }

    return ans;
}

Node* inOrderPredecessor(Node* root, Node* x) {
    Node* ans = NULL;
    while (root) {
        if (root->data >= x->data) {
            root = root->lchild;
        } else {
            ans = root;
            root = root->rchild;
        }
    }
    return ans;
}

```

```

Node* delete(Node* root, int key) {
    if (root == NULL) {
        return root;
    }

    if (key < root->data) {
        root->lchild = delete(root->lchild, key);
    } else if (key > root->data) {
        root->rchild = delete(root->rchild, key);
    } else {
        if (root->lchild == NULL) {
            Node* temp = root->rchild;
            free(root);
            return temp;
        } else if (root->rchild == NULL) {

```

```
    Node* temp = root->lchild;

    free(root);

    return temp;
}
```

```
    Node* temp = inOrderSuccessor(root, root);

    root->data = temp->data;

    root->rchild = delete(root->rchild, temp->data);
}

return root;
}
```

```
void inOrderTraversal(Node* p) {
    if (p != NULL) {
        inOrderTraversal(p->lchild);
        printf("%d ", p->data);
        inOrderTraversal(p->rchild);
    }
}
```

```
int main() {
    int choice, key;
    Node* temp;

    while (1) {
        printf("\nMenu:\n");
        printf("1. Create Binary Search Tree\n");
        printf("2. Search a Key\n");
        printf("3. Find In-Order Predecessor and Successor\n");
        printf("4. Insert a Key\n");
        printf("5. Delete a Key\n");
        printf("6. Display Tree (In-Order Traversal)\n");
    }
}
```

```

printf("7. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice) {
case 1:
    printf("Enter keys to insert (-1 to stop): ");
    while (1) {
        scanf("%d", &key);
        if (key == -1) break;
        root = insert(root, key);
    }
    break;
case 2:
    printf("Enter key to search: ");
    scanf("%d", &key);
    temp = search(root, key);
    if (temp != NULL) {
        printf("Key found: %d\n", temp->data);
    } else {
        printf("Key not found!\n");
    }
    break;
case 3:
    printf("Enter key to find predecessor and successor: ");
    scanf("%d", &key);
    temp = search(root, key);
    if (temp != NULL) {
        Node* pre = inOrderPredecessor(root, temp);
        Node* succ = inOrderSuccessor(root, temp);
        printf("Predecessor: %d, Successor: %d\n",
            pre ? pre->data : -1,

```

```

        succ ? succ->data : -1);
    } else {
        printf("Key not found!\n");
    }
    break;
case 4:
    printf("Enter key to insert: ");
    scanf("%d", &key);
    root = insert(root, key);
    break;
case 5:
    printf("Enter key to delete: ");
    scanf("%d", &key);
    root = delete(root, key);
    break;
case 6:
    printf("In-Order Traversal: ");
    inOrderTraversal(root);
    printf("\n");
    break;
case 7:
    exit(0);
default:
    printf("Invalid choice! Try again.\n");
}
}

return 0;
}

```

Q4.

Assignment 2:

Q1.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node {  
    int data;  
    struct node *next;  
};
```

```
struct node *first = NULL;
```

```
void create(int a[], int n) {  
    first = (struct node *)malloc(sizeof(struct node));  
    struct node *t;  
    struct node *last;  
    first->data = a[0];  
    first->next = NULL;  
    last = first;  
    for (int i = 1; i < n; i++) {  
        t = (struct node *)malloc(sizeof(struct node));  
        t->data = a[i];  
        t->next = last->next;  
        last->next = t;  
        last = t;  
    }  
}
```

```
void display(struct node *p) {  
    while (p != NULL) {
```

```

        printf("%d ", p->data);
        p = p->next;
    }
    printf("\n");
}

```

```

int count(struct node *p) {
    int c = 0;
    while (p != NULL) {
        c++;
        p = p->next;
    }
    return c;
}

```

```

void insert(struct node *p, int position, int data) {
    struct node *t = (struct node *)malloc(sizeof(struct node));
    if (position == 0) {
        t->data = data;
        t->next = first;
        first = t;
    } else {
        for (int i = 0; i < position - 1; i++) {
            p = p->next;
        }
        t->data = data;
        t->next = p->next;
        p->next = t;
    }
}

```

```

int deleteNode(struct node *p, int pos) {

```

```

struct node *q;

int x;

if (pos < 1 || pos > count(p)) {
    return -1;
}

if (pos == 1) {
    p = first;
    first = first->next;
    x = p->data;
    free(p);
} else {
    for (int i = 0; i < pos - 1; i++) {
        q = p;
        p = p->next;
    }
    q->next = p->next;
    x = p->data;
    free(p);
}

return x;
}

```

```

void reverse(struct node *p) {
    if (p == NULL) return;
    reverse(p->next);
    printf("%d ", p->data);
}

```

```

void lreverse(struct node *p) {
    struct node *q = NULL;
    struct node *r = NULL;
    while (p != NULL) {

```

```

        r = q;

        q = p;

        p = p->next;

        q->next = r;
    }
    first = q;
}

int main() {
    int choice, key, pos, n;

    printf("Enter the number of elements in the linked list: ");
    scanf("%d", &n);

    int a[n];

    printf("Enter the elements of the linked list: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    create(a, n);

    while (1) {
        printf("\nMenu:\n");
        printf("1. Insert a node at a specified position\n");
        printf("2. Delete a node from a specified position\n");
        printf("3. Count the number of nodes\n");
        printf("4. Reverse print the linked list\n");
        printf("5. Reverse the linked list\n");
        printf("6. Display the linked list\n");
        printf("7. Exit\n");
        printf("Enter your choice: ");
    }
}

```

```
scanf("%d", &choice);

switch (choice) {
case 1:
    printf("Enter position to insert: ");
    scanf("%d", &pos);
    printf("Enter value to insert: ");
    scanf("%d", &key);
    insert(first, pos, key);
    break;
case 2:
    printf("Enter position to delete: ");
    scanf("%d", &pos);
    key = deleteNode(first, pos);
    if (key != -1) {
        printf("Deleted node with value: %d\n", key);
    } else {
        printf("Invalid position!\n");
    }
    break;
case 3:
    printf("Number of nodes: %d\n", count(first));
    break;
case 4:
    printf("Reverse print of the linked list: ");
    reverse(first);
    printf("\n");
    break;
case 5:
    lreverse(first);
    printf("Linked list has been reversed.\n");
    break;
```

```

    case 6:
        printf("Linked list: ");
        display(first);
        break;
    case 7:
        exit(0);
    default:
        printf("Invalid choice! Try again.\n");
    }
}

return 0;
}

```

Q3.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```

struct node {
    int data;
    struct node *previous;
    struct node *next;
};

```

```
struct node *first = NULL;
```

```

void create(int a[], int n) {
    first = (struct node *)malloc(sizeof(struct node));
    struct node *last, *t;
    last = first;
    first->data = a[0];
    first->previous = NULL;
}

```

```

first->next = NULL;

for (int i = 1; i < n; i++) {
    t = (struct node *)malloc(sizeof(struct node));
    t->data = a[i];
    t->next = last->next;
    t->previous = last;
    last->next = t;
    last = t;
}
}

void display(struct node *p) {
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
    printf("\n");
}

int count(struct node *p) {
    int c = 0;
    while (p != NULL) {
        c++;
        p = p->next;
    }
    return c;
}

void insert(struct node *p, int pos, int data) {
    struct node *t = (struct node *)malloc(sizeof(struct node));
    if (pos < 0 || pos > count(p)) {
        printf("Enter a valid position to insert\n");
    }
}

```

```

        return;
    }
    if (pos == 0) {
        if (first == NULL) {
            t->data = data;
            t->next = t->previous = NULL;
            first = t;
        } else {
            t->data = data;
            t->next = first;
            first->previous = t;
            t->previous = NULL;
            first = t;
        }
    } else {
        for (int i = 0; i < pos - 1; i++) {
            p = p->next;
        }
        t->data = data;
        t->next = p->next;
        t->previous = p;
        if (p->next) {
            p->next->previous = t;
        }
        p->next = t;
    }
}

int deletenode(struct node *p, int pos) {
    int x;
    if (pos < 0 || pos >= count(p)) {
        printf("Enter a valid position to delete\n");
    }
}

```

```

        return -1;
    }
    if (pos == 0) {
        struct node *t = first;
        if (first->next != NULL) {
            first = first->next;
            first->previous = NULL;
            x = t->data;
            free(t);
        } else {
            first = NULL;
        }
    } else {
        for (int i = 0; i < pos - 1; i++) {
            p = p->next;
        }
        if (p->next) p->next->previous = p->previous;
        if (p->previous) p->previous->next = p->next;
        x = p->data;
        free(p);
    }
    return x;
}

```

```

void reverse(struct node *p) {
    if (p == NULL) return;
    reverse(p->next);
    printf("%d ", p->data);
}

```

```

void reverseprint(struct node *p) {
    while (p->next != NULL) {

```

```

        p = p->next;
    }
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->previous;
    }
    printf("\n");
}

int main() {
    int choice, key, pos, n;

    printf("Enter the number of elements in the doubly linked list: ");
    scanf("%d", &n);

    int *b = (int *)malloc(n * sizeof(int));

    printf("Enter the values for the linked list:\n");
    for (int i = 0; i < n; i++) {
        printf("Enter value for node %d: ", i + 1);
        scanf("%d", &b[i]);
    }

    create(b, n);

    while (1) {
        printf("\nMenu:\n");
        printf("1. Insert a node at a specified position\n");
        printf("2. Delete a node from a specified position\n");
        printf("3. Count the number of nodes in the linked list\n");
        printf("4. Reverse print the linked list\n");
        printf("5. Reverse the linked list\n");
    }
}

```

```
printf("6. Display the linked list\n");
printf("7. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice) {
case 1:
    printf("Enter position to insert: ");
    scanf("%d", &pos);
    printf("Enter value to insert: ");
    scanf("%d", &key);
    insert(first, pos, key);
    break;
case 2:
    printf("Enter position to delete: ");
    scanf("%d", &pos);
    key = deletenode(first, pos);
    if (key != -1) {
        printf("Deleted node with value: %d\n", key);
    } else {
        printf("Invalid position!\n");
    }
    break;
case 3:
    printf("Number of nodes: %d\n", count(first));
    break;
case 4:
    printf("Reverse print of the linked list: ");
    reverseprint(first);
    break;
case 5:
    printf("Reversing the linked list: ");
```

```

        reverse(first);

        printf("\n");

        break;
case 6:
    printf("Linked list: ");
    display(first);
    break;
case 7:
    free(b);
    exit(0);
default:
    printf("Invalid choice! Try again.\n");
}
}

return 0;
}

```

Q4.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```

struct Node {
    int data;
    struct Node *next;
} *top = NULL;

```

```

void push(int x) {
    struct Node *t;
    t = (struct Node*)malloc(sizeof(struct Node));

    if (t == NULL)

```

```
        printf("Stack is full\n");
    else {
        t->data = x;
        t->next = top;
        top = t;
    }
}
```

```
int pop() {
    struct Node *t;
    int x = -1;

    if (top == NULL)
        printf("Stack is empty\n");
    else {
        t = top;
        top = top->next;
        x = t->data;
        free(t);
    }
    return x;
}
```

```
int isEmpty() {
    return top == NULL;
}
```

```
void display() {
    struct Node *p;
    p = top;
    if (p == NULL) {
        printf("Stack is empty\n");
    }
}
```

```

    } else {
        while (p != NULL) {
            printf("%d ", p->data);
            p = p->next;
        }
        printf("\n");
    }
}

int main() {
    int choice, value;

    while (1) {
        printf("\nMenu:\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Check if Stack is Empty\n");
        printf("4. Display Stack\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &value);
                push(value);
                break;
            case 2:
                value = pop();
                if (value != -1) {
                    printf("Popped value: %d\n", value);
                }
            }
        }
    }
}

```

```

        }
        break;
case 3:
    if (isEmpty()) {
        printf("Stack is empty\n");
    } else {
        printf("Stack is not empty\n");
    }
    break;
case 4:
    display();
    break;
case 5:
    exit(0);
default:
    printf("Invalid choice! Try again.\n");
    }
}

return 0;
}

```

q5.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
    char data;
    struct Node *next;
} *top = NULL;

```

```
void push(char x) {  
    struct Node *t;  
    t = (struct Node*)malloc(sizeof(struct Node));  
  
    if (t == NULL)  
        printf("Stack is full\n");  
    else {  
        t->data = x;  
        t->next = top;  
        top = t;  
    }  
}
```

```
char pop() {  
    struct Node *t;  
    char x = -1;  
  
    if (top == NULL)  
        printf("Stack is Empty\n");  
    else {  
        t = top;  
        top = top->next;  
        x = t->data;  
        free(t);  
    }  
    return x;  
}
```

```
void Display() {  
    struct Node *p;  
    p = top;  
    while (p != NULL) {
```

```
    printf("%c ", p->data);  
    p = p->next;  
}  
printf("\n");  
}
```

```
int isBalanced(char *exp) {  
    int i;  
  
    for (i = 0; exp[i] != '\0'; i++) {  
        if (exp[i] == '(')  
            push(exp[i]);  
        else if (exp[i] == ')') {  
            if (top == NULL)  
                return 0;  
            pop();  
        }  
    }  
    return top == NULL ? 1 : 0;  
}
```

```
int pre(char x) {  
    if (x == '+' || x == '-')  
        return 1;  
    else if (x == '*' || x == '/')  
        return 2;  
    return 0;  
}
```

```
int isOperand(char x) {  
    if (x == '+' || x == '-' || x == '*' || x == '/')  
        return 0;
```

```

else
    return 1;
}

char *InToPost(char *infix) {
    int i = 0, j = 0;
    char *postfix;
    int len = strlen(infix);
    postfix = (char *)malloc((len + 2) * sizeof(char));

    while (infix[i] != '\0') {
        if (isOperand(infix[i]))
            postfix[j++] = infix[i++];
        else {
            if (pre(infix[i]) > pre(top->data))
                push(infix[i++]);
            else {
                postfix[j++] = pop();
            }
        }
    }

    while (top != NULL)
        postfix[j++] = pop();

    postfix[j] = '\0';
    return postfix;
}

```

```

int main() {
    char infix[100];
    printf("Enter the infix expression: ");
    scanf("%s", infix);
}

```

```
push('#');

char *postfix = InToPost(infix);

printf("Postfix expression: %s\n", postfix);

return 0;
}
```

q6.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
struct Node {
    int data;
    struct Node *next;
} *top = NULL;
```

```
void push(int x) {
    struct Node *t;
    t = (struct Node*)malloc(sizeof(struct Node));

    if (t == NULL)
        printf("Stack is full\n");
    else {
        t->data = x;
        t->next = top;
        top = t;
    }
}
```

```
int pop() {  
    struct Node *t;  
    int x = -1;  
  
    if (top == NULL)  
        printf("Stack is Empty\n");  
    else {  
        t = top;  
        top = top->next;  
        x = t->data;  
        free(t);  
    }  
    return x;  
}
```

```
void Display() {  
    struct Node *p;  
    p = top;  
    while (p != NULL) {  
        printf("%d ", p->data);  
        p = p->next;  
    }  
    printf("\n");  
}
```

```
int isOperand(char x) {  
    if (x == '+' || x == '-' || x == '*' || x == '/')  
        return 0;  
    else  
        return 1;  
}
```

```

int Eval(char *postfix) {
    int i = 0;
    int x1, x2, r = 0;

    for (i = 0; postfix[i] != '\0'; i++) {
        if (isOperand(postfix[i]))
            push(postfix[i] - '0');
        else {
            x2 = pop();
            x1 = pop();
            switch (postfix[i]) {
                case '+': r = x1 + x2; break;
                case '-': r = x1 - x2; break;
                case '*': r = x1 * x2; break;
                case '/': r = x1 / x2; break;
            }
            push(r);
        }
    }
    return top->data;
}

int main() {
    char postfix[100];

    printf("Enter the postfix expression: ");
    scanf("%s", postfix);

    printf("Result is %d\n", Eval(postfix));

    return 0;
}

```

```
}
```

q7.1

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct QueueArray {
```

```
    int size;
```

```
    int front;
```

```
    int rear;
```

```
    int *Q;
```

```
};
```

```
void createQueueArray(struct QueueArray *q, int size) {
```

```
    q->size = size;
```

```
    q->front = q->rear = -1;
```

```
    q->Q = (int *)malloc(q->size * sizeof(int));
```

```
}
```

```
void enqueueArray(struct QueueArray *q, int x) {
```

```
    if (q->rear == q->size - 1) {
```

```
        printf("Queue is Full\n");
```

```
    } else {
```

```
        q->rear++;
```

```
        q->Q[q->rear] = x;
```

```
    }
```

```
}
```

```
int dequeueArray(struct QueueArray *q) {
```

```
    int x = -1;
```

```
    if (q->front == q->rear) {
```

```

        printf("Queue is Empty\n");
    } else {
        q->front++;
        x = q->Q[q->front];
    }
    return x;
}

void displayQueueArray(struct QueueArray q) {
    for (int i = q.front + 1; i <= q.rear; i++) {
        printf("%d ", q.Q[i]);
    }
    printf("\n");
}

int countQueueArray(struct QueueArray q) {
    return q.rear - q.front;
}

int overflowQueueArray(struct QueueArray q) {
    return q.rear == q.size - 1;
}

int underflowQueueArray(struct QueueArray q) {
    return q.rear == q.front;
}

void freeQueueArray(struct QueueArray *q) {
    free(q->Q);
}

int main() {

```

```
struct QueueArray qArray;

int choice, value, size;

printf("Enter the size of the queue for array: ");
scanf("%d", &size);
createQueueArray(&qArray, size);

do {
    printf("\nMenu:\n");
    printf("1. Enqueue (Array)\n");
    printf("2. Dequeue (Array)\n");
    printf("3. Display (Array)\n");
    printf("4. Count elements (Array)\n");
    printf("5. Check overflow (Array)\n");
    printf("6. Check underflow (Array)\n");
    printf("7. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            printf("Enter a value to enqueue (Array): ");
            scanf("%d", &value);
            enqueueArray(&qArray, value);
            break;
        case 2:
            value = dequeueArray(&qArray);
            if (value != -1) {
                printf("Dequeued value (Array): %d\n", value);
            }
            break;
        case 3:
```

```
        displayQueueArray(qArray);
        break;
case 4:
    printf("Number of elements in the queue (Array): %d\n", countQueueArray(qArray));
    break;
case 5:
    if (overflowQueueArray(qArray)) {
        printf("Queue is full (Array)\n");
    } else {
        printf("Queue is not full (Array)\n");
    }
    break;
case 6:
    if (underflowQueueArray(qArray)) {
        printf("Queue is empty (Array)\n");
    } else {
        printf("Queue is not empty (Array)\n");
    }
    break;
case 7:
    printf("Exiting program...\n");
    break;
default:
    printf("Invalid choice! Please try again.\n");
    break;
    }
} while (choice != 7);

freeQueueArray(&qArray);
return 0;
}
```

Q7.2

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

```
struct Queue {  
    struct Node *front, *rear;  
};
```

```
void createQueue(struct Queue *q) {  
    q->front = q->rear = NULL;  
}
```

```
void enqueue(struct Queue *q, int x) {  
    struct Node *t;  
    t = (struct Node *)malloc(sizeof(struct Node));
```

```
    if (t == NULL) {  
        printf("Queue is Full\n");  
    } else {  
        t->data = x;  
        t->next = NULL;  
        if (q->front == NULL) {  
            q->front = q->rear = t;  
        } else {  
            q->rear->next = t;  
            q->rear = t;  
        }  
    }
```

```
    }  
}
```

```
int dequeue(struct Queue *q) {  
    int x = -1;  
    struct Node *t;  
  
    if (q->front == NULL) {  
        printf("Queue is Empty\n");  
    } else {  
        x = q->front->data;  
        t = q->front;  
        q->front = q->front->next;  
        free(t);  
    }  
    return x;  
}
```

```
void displayQueue(struct Queue q) {  
    struct Node *p = q.front;  
    if (p == NULL) {  
        printf("Queue is Empty\n");  
        return;  
    }  
    while (p) {  
        printf("%d ", p->data);  
        p = p->next;  
    }  
    printf("\n");  
}
```

```
int countElements(struct Queue q) {
```

```
int count = 0;
struct Node *p = q.front;
while (p) {
    count++;
    p = p->next;
}
return count;
}
```

```
int isOverflow(struct Queue q) {
    struct Node *p = q.front;
    int size = 0;
    while (p) {
        size++;
        p = p->next;
    }
    return size == 100;
}
```

```
int isUnderflow(struct Queue q) {
    return q.front == NULL;
}
```

```
int main() {
    struct Queue q;
    createQueue(&q);
    int choice, value;

    do {
        printf("\nMenu:\n");
        printf("1. Construct a linear queue\n");
        printf("2. Insertion (Enqueue)\n");
```

```
printf("3. Deletion (Dequeue)\n");
printf("4. Display queue\n");
printf("5. Count number of elements\n");
printf("6. Check overflow condition\n");
printf("7. Check underflow condition\n");
printf("8. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice) {
    case 1:
        createQueue(&q);
        printf("Queue is constructed.\n");
        break;
    case 2:
        printf("Enter value to enqueue: ");
        scanf("%d", &value);
        enqueue(&q, value);
        break;
    case 3:
        value = dequeue(&q);
        if (value != -1) {
            printf("Dequeued value: %d\n", value);
        }
        break;
    case 4:
        printf("Queue: ");
        displayQueue(q);
        break;
    case 5:
        printf("Number of elements in queue: %d\n", countElements(q));
        break;
```

```

        case 6:
            if (isOverflow(q)) {
                printf("Queue is full.\n");
            } else {
                printf("Queue is not full.\n");
            }
            break;
        case 7:
            if (isUnderflow(q)) {
                printf("Queue is empty.\n");
            } else {
                printf("Queue is not empty.\n");
            }
            break;
        case 8:
            printf("Exiting program...\n");
            break;
        default:
            printf("Invalid choice! Please try again.\n");
            break;
    }
} while (choice != 8);

return 0;
}

```

Q8

```

#include <stdio.h>
#include <stdlib.h>

```

```

struct Queue {
    int size;
    int front;

```

```
int rear;

int *Q;

};

void create(struct Queue *q, int size) {
    q->size = size;
    q->front = q->rear = 0;
    q->Q = (int *)malloc(q->size * sizeof(int));
}
```

```
void enqueue(struct Queue *q, int x) {
    if ((q->rear + 1) % q->size == q->front) {
        printf("Queue is Full\n");
    } else {
        q->rear = (q->rear + 1) % q->size;
        q->Q[q->rear] = x;
    }
}
```

```
int dequeue(struct Queue *q) {
    int x = -1;
    if (q->front == q->rear) {
        printf("Queue is Empty\n");
    } else {
        q->front = (q->front + 1) % q->size;
        x = q->Q[q->front];
    }
    return x;
}
```

```
void Display(struct Queue q) {
    int i = q.front + 1;
```

```
do {  
    printf("%d ", q.Q[i]);  
    i = (i + 1) % q.size;  
} while (i != (q.rear + 1) % q.size);  
printf("\n");  
}
```

```
int count(struct Queue q) {  
    int c = 0;  
    int i = q.front + 1;  
    do {  
        c++;  
        i = (i + 1) % q.size;  
    } while (i != (q.rear + 1) % q.size);  
    return c;  
}
```

```
int overflow(struct Queue q) {  
    return (q.rear + 1) % q.size == q.front;  
}
```

```
int underflow(struct Queue q) {  
    return q.rear == q.front;  
}
```

```
int main() {  
    struct Queue q;  
    int choice, value;  
  
    printf("Enter the size of the queue: ");  
    scanf("%d", &q.size);  
    create(&q, q.size + 1); // Added 1 to handle circular queue behavior
```

```
do {  
    printf("\nMenu:\n");  
    printf("1. Enqueue\n");  
    printf("2. Dequeue\n");  
    printf("3. Display\n");  
    printf("4. Count elements\n");  
    printf("5. Check if queue is full\n");  
    printf("6. Check if queue is empty\n");  
    printf("7. Exit\n");  
    printf("Enter your choice: ");  
    scanf("%d", &choice);  
  
    switch (choice) {  
        case 1:  
            printf("Enter a value to enqueue: ");  
            scanf("%d", &value);  
            enqueue(&q, value);  
            break;  
  
        case 2:  
            value = dequeue(&q);  
            if (value != -1) {  
                printf("Dequeued value: %d\n", value);  
            }  
            break;  
  
        case 3:  
            Display(q);  
            break;  
  
        case 4:
```

```
    printf("Number of elements in the queue: %d\n", count(q));  
    break;  
  
case 5:  
    if (overflow(q)) {  
        printf("Queue is full\n");  
    } else {  
        printf("Queue is not full\n");  
    }  
    break;  
  
case 6:  
    if (underflow(q)) {  
        printf("Queue is empty\n");  
    } else {  
        printf("Queue is not empty\n");  
    }  
    break;  
  
case 7:  
    printf("Exiting program...\n");  
    break;  
  
default:  
    printf("Invalid choice! Please try again.\n");  
    break;  
}  
} while (choice != 7);  
  
free(q.Q);  
return 0;  
}
```

q9

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct stack {
```

```
    int top;
```

```
    int *p;
```

```
    int size;
```

```
};
```

```
void create(struct stack *st) {
```

```
    printf("Enter the size of the stack: ");
```

```
    scanf("%d", &st->size);
```

```
    st->p = (int *)malloc(st->size * sizeof(int));
```

```
    st->top = -1;
```

```
}
```

```
int isempty(struct stack *s) {
```

```
    return (s->top == -1);
```

```
}
```

```
int isfull(struct stack *s) {
```

```
    return (s->top == s->size - 1);
```

```
}
```

```
void push(struct stack *st, int x) {
```

```
    if (isfull(st)) {
```

```
        printf("Stack overflow\n");
```

```
    } else {
```

```
        st->top++;
```

```
        st->p[st->top] = x;
```

```
    }  
}
```

```
int pop(struct stack *st) {  
    int x = -1;  
    if (isempty(st)) {  
        printf("Stack underflow\n");  
    } else {  
        x = st->p[st->top];  
        st->top--;  
    }  
    return x;  
}
```

```
void display(struct stack *s1, struct stack *s2) {  
    if (isempty(s1) && isempty(s2)) {  
        printf("Queue is empty\n");  
        return;  
    }  
  
    while (!isempty(s1)) {  
        int x = pop(s1);  
        push(s2, x);  
    }  
  
    for (int i = s2->top; i >= 0; i--) {  
        printf("%d ", s2->p[i]);  
    }  
    printf("\n");  
  
    while (!isempty(s2)) {  
        int x = pop(s2);
```

```

        push(s1, x);
    }
}

void enqueue(struct stack *s1, int data) {
    push(s1, data);
}

void dequeue(struct stack *s1, struct stack *s2) {
    if (isempty(s2)) {
        if (isempty(s1)) {
            printf("Queue is empty\n");
        } else {
            while (!isempty(s1)) {
                int x = pop(s1);
                push(s2, x);
            }
        }
    }

    if (!isempty(s2)) {
        int x = pop(s2);
        printf("Dequeued element: %d\n", x);
    }
}

int main() {
    struct stack st1, st2;
    create(&st1);
    create(&st2);
    int choice, value;

```

```
do {  
    printf("\nMenu:\n");  
    printf("1. Enqueue\n");  
    printf("2. Dequeue\n");  
    printf("3. Display Queue\n");  
    printf("4. Exit\n");  
    printf("Enter your choice: ");  
    scanf("%d", &choice);  
  
    switch (choice) {  
        case 1:  
            printf("Enter a value to enqueue: ");  
            scanf("%d", &value);  
            enqueue(&st1, value);  
            break;  
  
        case 2:  
            dequeue(&st1, &st2);  
            break;  
  
        case 3:  
            display(&st1, &st2);  
            break;  
  
        case 4:  
            printf("Exiting program...\n");  
            break;  
  
        default:  
            printf("Invalid choice! Please try again.\n");  
            break;  
    }  
}
```

```
    } while (choice != 4);

    free(st1.p);
    free(st2.p);
    return 0;
}
```

Assignment 0:

Q1.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
struct Student {
    char name[50];
    char address[100];
    int marks[5];
    int total;
    float average;
};
```

```
void inputStudentDetails(struct Student *s) {
    printf("Enter student's name: ");
    scanf("%s", s->name);

    printf("Enter student's address: ");
    scanf("%s", s->address);

    printf("Enter marks for 5 subjects (Mathematics, Physics, Chemistry, Biology, English):\n");
    s->total = 0;
    for (int i = 0; i < 5; i++) {
```

```

        printf("Subject %d: ", i + 1);
        scanf("%d", &s->marks[i]);
        s->total += s->marks[i];
    }
    s->average = s->total / 5.0;
}

int checkPassOrFail(struct Student *s) {
    int failCount = 0;
    for (int i = 0; i < 5; i++) {
        if (s->marks[i] < 40) {
            failCount++;
        }
    }
    return failCount >= 3;
}

void printStudentRecord(struct Student *s) {
    printf("Name: %s\n", s->name);
    printf("Address: %s\n", s->address);
    printf("Marks: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", s->marks[i]);
    }
    printf("\nTotal: %d\n", s->total);
    printf("Average: %.2f\n", s->average);
}

int main() {
    struct Student students[12];

    struct Student *maxMathStudent = NULL, *minPhysicsStudent = NULL, *maxAvgStudent =
    NULL;

    int studentCount = 0, passed, maxMarksMath = -1, minMarksPhysics = 101;

```

```

int i, choice;

do {
    printf("\nMenu:\n");
    printf("1. Input Student Details\n");
    printf("2. Display Student with Maximum Marks in Mathematics\n");
    printf("3. Display Student with Minimum Marks in Physics\n");
    printf("4. Display Student with Maximum Average Marks\n");
    printf("5. Display All Students' Records with Total Marks\n");
    printf("6. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            if (studentCount < 12) {
                printf("\nEnter details for student %d:\n", studentCount + 1);
                inputStudentDetails(&students[studentCount]);
                studentCount++;
            } else {
                printf("Student limit reached.\n");
            }
            break;
        case 2:
            if (studentCount > 0) {
                for (i = 0; i < studentCount; i++) {
                    if (students[i].marks[0] > maxMarksMath) {
                        maxMarksMath = students[i].marks[0];
                        maxMathStudent = &students[i];
                    }
                }
                printf("\nStudent with the maximum marks in Mathematics:\n");
            }
        }
    }

```

```

        printStudentRecord(maxMathStudent);
    } else {
        printf("No students available.\n");
    }
    break;
case 3:
    if (studentCount > 0) {
        for (i = 0; i < studentCount; i++) {
            if (students[i].marks[1] < minMarksPhysics) {
                minMarksPhysics = students[i].marks[1];
                minPhysicsStudent = &students[i];
            }
        }
        printf("\nStudent with the minimum marks in Physics:\n");
        printStudentRecord(minPhysicsStudent);
    } else {
        printf("No students available.\n");
    }
    break;
case 4:
    if (studentCount > 0) {
        for (i = 0; i < studentCount; i++) {
            if (maxAvgStudent == NULL || students[i].average > maxAvgStudent->average) {
                maxAvgStudent = &students[i];
            }
        }
        printf("\nStudent with the maximum average marks:\n");
        printStudentRecord(maxAvgStudent);
    } else {
        printf("No students available.\n");
    }
    break;

```

```

case 5:
    if (studentCount > 0) {
        printf("\nAll students' records with total marks:\n");
        for (i = 0; i < studentCount; i++) {
            printf("Name: %s, Total: %d\n", students[i].name, students[i].total);
        }
    } else {
        printf("No students available.\n");
    }
    break;
case 6:
    printf("Exiting program...\n");
    break;
default:
    printf("Invalid choice! Please try again.\n");
}
} while (choice != 6);

return 0;
}

```

q2

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

```

#define MAX_ACCOUNTS 100
#define MIN_BALANCE 1000
#define INTEREST_RATE 0.04

```

```

struct SavingsAccount {
    char name[50];

```

```
    int accountNumber;

    float balance;
};

struct SavingsAccount accounts[MAX_ACCOUNTS];
int accountCount = 0;

void createAccount() {
    if (accountCount >= MAX_ACCOUNTS) {
        printf("Account limit reached!\n");
        return;
    }

    struct SavingsAccount newAccount;
    newAccount.accountNumber = accountCount + 1;

    printf("Enter account holder's name: ");
    scanf("%s", newAccount.name);

    printf("Enter initial deposit (minimum Rs. 1000): ");
    scanf("%f", &newAccount.balance);

    if (newAccount.balance < MIN_BALANCE) {
        printf("Initial deposit must be at least Rs. %d\n", MIN_BALANCE);
        return;
    }

    accounts[accountCount] = newAccount;
    accountCount++;

    printf("Account created successfully! Account number: %d\n", newAccount.accountNumber);
}
```

```
void deposit(int accountNumber) {  
    if (accountNumber < 1 || accountNumber > accountCount) {  
        printf("Invalid account number!\n");  
        return;  
    }  
  
    float amount;  
    printf("Enter deposit amount: ");  
    scanf("%f", &amount);  
  
    if (amount <= 0) {  
        printf("Amount should be positive.\n");  
        return;  
    }  
  
    accounts[accountNumber - 1].balance += amount;  
    printf("Deposit successful! New balance: Rs. %.2f\n", accounts[accountNumber - 1].balance);  
}
```

```
void withdraw(int accountNumber) {  
    if (accountNumber < 1 || accountNumber > accountCount) {  
        printf("Invalid account number!\n");  
        return;  
    }  
  
    float amount;  
    printf("Enter withdrawal amount: ");  
    scanf("%f", &amount);  
  
    if (amount <= 0) {  
        printf("Amount should be positive.\n");  
        return;  
    }  
}
```

```

    }

    if (accounts[accountNumber - 1].balance - amount < MIN_BALANCE) {
        printf("Insufficient balance! Minimum balance of Rs. %.2f is required.\n", MIN_BALANCE);
        return;
    }

    accounts[accountNumber - 1].balance -= amount;
    printf("Withdrawal successful! New balance: Rs. %.2f\n", accounts[accountNumber - 1].balance);
}

void calculateInterest() {
    float totalInterest = 0;
    int i;

    for (i = 0; i < accountCount; i++) {
        float interest = accounts[i].balance * INTEREST_RATE;
        accounts[i].balance += interest;
        totalInterest += interest;
        printf("Account %d (%s) interest earned: Rs. %.2f. New balance: Rs. %.2f\n",
            accounts[i].accountNumber, accounts[i].name, interest, accounts[i].balance);
    }
    printf("Total interest paid to all accounts: Rs. %.2f\n", totalInterest);
}

void displayAccountDetails(int accountNumber) {
    if (accountNumber < 1 || accountNumber > accountCount) {
        printf("Invalid account number!\n");
        return;
    }

    printf("\nAccount Number: %d\n", accounts[accountNumber - 1].accountNumber);
}

```

```
    printf("Account Holder's Name: %s\n", accounts[accountNumber - 1].name);  
    printf("Account Balance: Rs. %.2f\n", accounts[accountNumber - 1].balance);  
}
```

```
void displayAllAccounts() {  
    if (accountCount == 0) {  
        printf("No accounts available.\n");  
        return;  
    }  
  
    int i;  
    printf("\nAll Accounts Details:\n");  
    for (i = 0; i < accountCount; i++) {  
        printf("Account Number: %d\n", accounts[i].accountNumber);  
        printf("Account Holder: %s\n", accounts[i].name);  
        printf("Balance: Rs. %.2f\n", accounts[i].balance);  
        printf("-----\n");  
    }  
}
```

```
int main() {  
    int choice, accountNumber;  
  
    do {  
        printf("\nMenu:\n");  
        printf("1. Create Account\n");  
        printf("2. Deposit\n");  
        printf("3. Withdraw\n");  
        printf("4. Calculate Interest\n");  
        printf("5. Display Account Details\n");  
        printf("6. Display All Accounts\n");  
        printf("7. Exit\n");
```

```
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice) {
    case 1:
        createAccount();
        break;
    case 2:
        printf("Enter account number to deposit: ");
        scanf("%d", &accountNumber);
        deposit(accountNumber);
        break;
    case 3:
        printf("Enter account number to withdraw: ");
        scanf("%d", &accountNumber);
        withdraw(accountNumber);
        break;
    case 4:
        calculateInterest();
        break;
    case 5:
        printf("Enter account number to view details: ");
        scanf("%d", &accountNumber);
        displayAccountDetails(accountNumber);
        break;
    case 6:
        displayAllAccounts();
        break;
    case 7:
        printf("Exiting program...\n");
        break;
    default:
```

```
        printf("Invalid choice! Please try again.\n");
    }
} while (choice != 7);

return 0;
}
```

Assignment 1:

Q3.

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Term {
    int coeff;
    int exp;
};
```

```
struct Poly {
    int n;
    struct Term *terms;
};
```

```
void create(struct Poly *p) {
    int i;
    printf("Number of terms? ");
    scanf("%d", &p->n);
    p->terms = (struct Term*)malloc(p->n * sizeof(struct Term));

    printf("Enter terms (coefficient and exponent):\n");
    for (i = 0; i < p->n; i++) {
        scanf("%d %d", &p->terms[i].coeff, &p->terms[i].exp);
    }
}
```

```
}
```

```
void display(struct Poly p) {  
    int i;  
    for (i = 0; i < p.n; i++) {  
        printf("%dx^%d", p.terms[i].coeff, p.terms[i].exp);  
        if (i < p.n - 1) {  
            printf(" + ");  
        }  
    }  
    printf("\n");  
}
```

```
void sortPoly(struct Poly *p) {  
    int i, j;  
    struct Term temp;  
    for (i = 0; i < p->n - 1; i++) {  
        for (j = i + 1; j < p->n; j++) {  
            if (p->terms[i].exp < p->terms[j].exp) {  
                temp = p->terms[i];  
                p->terms[i] = p->terms[j];  
                p->terms[j] = temp;  
            }  
        }  
    }  
}
```

```
struct Poly *add(struct Poly *p1, struct Poly *p2) {  
    int i, j, k;  
    struct Poly *sum;  
  
    sum = (struct Poly*)malloc(sizeof(struct Poly));
```

```
sum->terms = (struct Term*)malloc((p1->n + p2->n) * sizeof(struct Term));
```

```
i = j = k = 0;
```

```
while (i < p1->n && j < p2->n) {
```

```
    if (p1->terms[i].exp > p2->terms[j].exp) {
```

```
        sum->terms[k++] = p1->terms[i++];
```

```
    } else if (p1->terms[i].exp < p2->terms[j].exp) {
```

```
        sum->terms[k++] = p2->terms[j++];
```

```
    } else {
```

```
        sum->terms[k].exp = p1->terms[i].exp;
```

```
        sum->terms[k].coeff = p1->terms[i].coeff + p2->terms[j].coeff;
```

```
        k++;
```

```
        i++;
```

```
        j++;
```

```
    }
```

```
}
```

```
while (i < p1->n) {
```

```
    sum->terms[k++] = p1->terms[i++];
```

```
}
```

```
while (j < p2->n) {
```

```
    sum->terms[k++] = p2->terms[j++];
```

```
}
```

```
sum->n = k;
```

```
sortPoly(sum); // Sort terms in descending order of exponents
```

```
return sum;
```

```
}
```

```
int main() {
```

```
    struct Poly p1, p2, *p3;
```

```
create(&p1);
create(&p2);

p3 = add(&p1, &p2);

printf("\nPolynomial 1: ");
display(p1);
printf("\nPolynomial 2: ");
display(p2);
printf("\nSum of Polynomials: ");
display(*p3);

free(p1.terms);
free(p2.terms);
free(p3->terms);
free(p3);

return 0;
}
```

Q4.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Term {
    int coeff;
    int exp;
};
```

```
struct Poly {
    int n;
```

```

    struct Term *terms;
};

void create(struct Poly *p) {
    int i;
    printf("Enter number of terms: ");
    scanf("%d", &p->n);
    p->terms = (struct Term*)malloc(p->n * sizeof(struct Term));

    printf("Enter terms (coefficient and exponent):\n");
    for (i = 0; i < p->n; i++) {
        scanf("%d %d", &p->terms[i].coeff, &p->terms[i].exp);
    }
}

```

```

void display(struct Poly p) {
    int i;
    for (i = 0; i < p.n; i++) {
        printf("%dx^%d", p.terms[i].coeff, p.terms[i].exp);
        if (i < p.n - 1) {
            printf(" + ");
        }
    }
    printf("\n");
}

```

```

int evaluate(struct Poly p, int x) {
    int result = 0;
    int i;
    for (i = 0; i < p.n; i++) {
        int termResult = p.terms[i].coeff;
        int exp = p.terms[i].exp;
    }
}

```

```

        for (int j = 0; j < exp; j++) {
            termResult *= x;
        }
        result += termResult;
    }
    return result;
}

int main() {
    struct Poly p;
    int x, result;

    create(&p);

    printf("Enter the value of x to evaluate the polynomial: ");
    scanf("%d", &x);

    result = evaluate(p, x);

    printf("The result of evaluating the polynomial at x = %d is: %d\n", x, result);

    free(p.terms);

    return 0;
}

```

Q5.

```

#include <stdio.h>
#include <stdlib.h>

```

```

struct Term {
    int coeff;

```

```

    int exp;
};

struct Poly {
    int n;
    struct Term *terms;
};

void create(struct Poly *p) {
    int i;
    printf("Enter number of terms: ");
    scanf("%d", &p->n);
    p->terms = (struct Term*)malloc(p->n * sizeof(struct Term));

    printf("Enter terms (coefficient and exponent):\n");
    for (i = 0; i < p->n; i++) {
        scanf("%d %d", &p->terms[i].coeff, &p->terms[i].exp);
    }
}

void display(struct Poly p) {
    int i;
    for (i = 0; i < p.n; i++) {
        printf("%dx^%d", p.terms[i].coeff, p.terms[i].exp);
        if (i < p.n - 1) {
            printf(" + ");
        }
    }
    printf("\n");
}

struct Poly* multiply(struct Poly *p1, struct Poly *p2) {

```

```

int i, j, k = 0;
struct Poly *prod = (struct Poly*)malloc(sizeof(struct Poly));
prod->terms = (struct Term*)malloc(p1->n * p2->n * sizeof(struct Term));

for (i = 0; i < p1->n; i++) {
    for (j = 0; j < p2->n; j++) {
        int coeff = p1->terms[i].coeff * p2->terms[j].coeff;
        int exp = p1->terms[i].exp + p2->terms[j].exp;

        int found = 0;
        for (int m = 0; m < k; m++) {
            if (prod->terms[m].exp == exp) {
                prod->terms[m].coeff += coeff;
                found = 1;
                break;
            }
        }

        if (!found) {
            prod->terms[k].coeff = coeff;
            prod->terms[k].exp = exp;
            k++;
        }
    }
}

prod->n = k;
return prod;
}

int main() {
    struct Poly p1, p2, *p3;

```

```
create(&p1);
create(&p2);

p3 = multiply(&p1, &p2);

printf("Polynomial 1: ");
display(p1);

printf("Polynomial 2: ");
display(p2);

printf("Product of Polynomials: ");
display(*p3);

free(p1.terms);
free(p2.terms);
free(p3->terms);
free(p3);

return 0;
}
```

Q6.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
```

```
void reverseString(char* str) {
    int len = strlen(str);
    int i;
    for (i = 0; i < len / 2; i++) {
```

```

        char temp = str[i];
        str[i] = str[len - i - 1];
        str[len - i - 1] = temp;
    }
}

char* addLargeNumbers(char* num1, char* num2) {
    int len1 = strlen(num1);
    int len2 = strlen(num2);
    int maxLen = len1 > len2 ? len1 : len2;

    char* result = (char*)malloc(maxLen + 2);
    int carry = 0, sum = 0, i = 0, j = 0, k = 0;

    reverseString(num1);
    reverseString(num2);

    while (i < len1 || j < len2 || carry) {
        int digit1 = i < len1 ? num1[i] - '0' : 0;
        int digit2 = j < len2 ? num2[j] - '0' : 0;
        sum = digit1 + digit2 + carry;
        result[k] = (sum % 10) + '0';
        carry = sum / 10;
        i++;
        j++;
        k++;
    }
    result[k] = '\0';

    reverseString(result);
    return result;
}

```

```
int main() {  
    char num1[1000], num2[1000];  
    char* result;  
  
    printf("Enter the first large number: ");  
    scanf("%s", num1);  
  
    printf("Enter the second large number: ");  
    scanf("%s", num2);  
  
    result = addLargeNumbers(num1, num2);  
  
    printf("Sum: %s\n", result);  
  
    free(result);  
  
    return 0;  
}
```

Q7.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Element {  
    int i;  
    int j;  
    int x;  
};
```

```
struct Sparse {  
    int m;
```

```

    int n;

    int num;

    struct Element *ele;
};

void create(struct Sparse *s) {
    int i;

    printf("Enter dimensions (rows and columns): ");
    scanf("%d%d", &s->m, &s->n);

    printf("Enter number of non-zero elements: ");
    scanf("%d", &s->num);

    s->ele = (struct Element *)malloc(s->num * sizeof(struct Element));

    printf("Enter non-zero elements (row, column, value):\n");
    for (i = 0; i < s->num; i++) {
        scanf("%d%d%d", &s->ele[i].i, &s->ele[i].j, &s->ele[i].x);
    }
}

void display(struct Sparse s) {
    int i, j, k = 0;

    for (i = 0; i < s.m; i++) {
        for (j = 0; j < s.n; j++) {
            if (k < s.num && i == s.ele[k].i && j == s.ele[k].j) {
                printf("%d ", s.ele[k++].x);
            } else {
                printf("0 ");
            }
        }
        printf("\n");
    }
}

```

```
}
```

```
struct Sparse *add(struct Sparse *s1, struct Sparse *s2) {
```

```
    struct Sparse *sum;
```

```
    int i = 0, j = 0, k = 0;
```

```
    if (s1->m != s2->m || s1->n != s2->n) {
```

```
        printf("Matrices cannot be added (different dimensions).\n");
```

```
        return NULL;
```

```
    }
```

```
    sum = (struct Sparse *)malloc(sizeof(struct Sparse));
```

```
    sum->ele = (struct Element *)malloc((s1->num + s2->num) * sizeof(struct Element));
```

```
    sum->m = s1->m;
```

```
    sum->n = s1->n;
```

```
    while (i < s1->num && j < s2->num) {
```

```
        if (s1->ele[i].i < s2->ele[j].i) {
```

```
            sum->ele[k++] = s1->ele[i++];
```

```
        } else if (s1->ele[i].i > s2->ele[j].i) {
```

```
            sum->ele[k++] = s2->ele[j++];
```

```
        } else {
```

```
            if (s1->ele[i].j < s2->ele[j].j) {
```

```
                sum->ele[k++] = s1->ele[i++];
```

```
            } else if (s1->ele[i].j > s2->ele[j].j) {
```

```
                sum->ele[k++] = s2->ele[j++];
```

```
            } else {
```

```
                sum->ele[k] = s1->ele[i];
```

```
                sum->ele[k++].x = s1->ele[i++].x + s2->ele[j++].x;
```

```
            }
```

```
        }
```

```
    }
```

```
while (i < s1->num) {  
    sum->ele[k++] = s1->ele[i++];  
}
```

```
while (j < s2->num) {  
    sum->ele[k++] = s2->ele[j++];  
}
```

```
sum->num = k;
```

```
return sum;  
}
```

```
int main() {  
    struct Sparse s1, s2, *s3;  
  
    printf("Create first sparse matrix:\n");  
    create(&s1);  
  
    printf("Create second sparse matrix:\n");  
    create(&s2);  
  
    s3 = add(&s1, &s2);  
  
    printf("First Matrix:\n");  
    display(s1);  
  
    printf("Second Matrix:\n");  
    display(s2);  
  
    if (s3) {
```

```
        printf("Sum Matrix:\n");
        display(*s3);
    }

    free(s1.ele);
    free(s2.ele);
    if (s3) {
        free(s3->ele);
        free(s3);
    }

    return 0;
}
```

Q8.

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Element {
    int i;
    int j;
    int x;
};
```

```
struct Sparse {
    int m;
    int n;
    int num;
    struct Element *ele;
};
```

```
void create(struct Sparse *s) {
```

```

int i;

printf("Enter dimensions (rows and columns): ");
scanf("%d%d", &s->m, &s->n);

printf("Enter number of non-zero elements: ");
scanf("%d", &s->num);

s->ele = (struct Element *)malloc(s->num * sizeof(struct Element));
printf("Enter non-zero elements (row, column, value):\n");
for (i = 0; i < s->num; i++) {
    scanf("%d%d%d", &s->ele[i].i, &s->ele[i].j, &s->ele[i].x);
}
}

```

```

void display(struct Sparse s) {
    int i, j, k = 0;
    for (i = 0; i < s.m; i++) {
        for (j = 0; j < s.n; j++) {
            if (k < s.num && i == s.ele[k].i && j == s.ele[k].j) {
                printf("%d ", s.ele[k++].x);
            } else {
                printf("0 ");
            }
        }
        printf("\n");
    }
}

```

```

void countNonZeroPerRow(struct Sparse s) {
    int *count = (int *)calloc(s.m, sizeof(int));
    int i;

    for (i = 0; i < s.num; i++) {

```

```

        count[s.ele[i].i]++;
    }

    printf("Number of non-zero elements in each row:\n");
    for (i = 0; i < s.m; i++) {
        printf("Row %d: %d\n", i, count[i]);
    }

    free(count);
}

int main() {
    struct Sparse s;

    create(&s);

    printf("Sparse Matrix:\n");
    display(s);

    countNonZeroPerRow(s);

    free(s.ele);

    return 0;
}

```

q9.

```
#include <stdio.h>
```

```

void print1Dfrom2D(int *arr, int rows, int cols) {
    int i, j;
    printf("Elements of the 1D array using 2D indices:\n");

```

```
    for (i = 0; i < rows; i++) {  
        for (j = 0; j < cols; j++) {  
            int k = i * cols + j; // Map 2D indices to 1D index  
            printf("Element at A[%d][%d]: %d\n", i, j, arr[k]);  
        }  
    }  
}  
  
int main() {  
    int rows, cols, i;  
    printf("Enter the number of rows and columns for the 2D array: ");  
    scanf("%d%d", &rows, &cols);  
  
    int size = rows * cols;  
    int arr[size];  
  
    printf("Enter the elements of the 1D array (size %d):\n", size);  
    for (i = 0; i < size; i++) {  
        scanf("%d", &arr[i]);  
    }  
  
    print1Dfrom2D(arr, rows, cols);  
  
    return 0;  
}
```